

SKILLS ACCELERATOR

Cloud Platform Complete Project Documentation

The complete Week 1 to Week 4 engineering journey: application development, containerisation, AWS infrastructure, CI/CD automation, security hardening, monitoring, scaling, cost, and lessons learned.

PLATFORM

Node.js + PostgreSQL + Docker

REGION

Africa (Cape Town) - af-south-1

INFRASTRUCTURE

Terraform with AWS Provider ~> 6.0

DOCUMENT VERSION

1.0 | 22 August 2026

Scope: the full four-week project lifecycle and the final repository configuration reviewed on 22 August 2026.

SECTION 01

Project overview

The complete Week 1 to Week 4 delivery journey

Skills Accelerator is a full-stack CRUD web application for publishing and discovering skills. It began in Week 1 as a local Node.js, Express, and PostgreSQL application, then progressed through Docker containerisation, Terraform-managed AWS infrastructure, automated delivery, and finally operational security and monitoring. The final result is one end-to-end cloud engineering project rather than four independent exercises.

APPLICATION Node.js 22 Express 5 API on port 3000	DATABASE PostgreSQL 17 Amazon RDS, 20 GB storage	COMPUTE EC2 t3.micro Auto Scaling Group: 1 to 2
INGRESS Public ALB HTTP listener on port 80	OPERATIONS SSM + CloudWatch No inbound SSH; central logs	DELIVERY GitHub + Docker Hub Build and publish on main

Four-week progression

Phase	Focus	Delivered outcome
Week 1	Application and containerisation	CRUD web application, PostgreSQL persistence, Docker image, Docker Hub
Week 2	Infrastructure as Code and AWS	VPC, subnets, ALB, private EC2 Auto Scaling, RDS, IAM, Session Manager
Week 3	CI/CD and deployment automation	GitHub Actions build, Docker Hub publish, deployment workflow, scaling policies
Week 4	Security, monitoring, and final hardening	Parameter Store, least-privilege IAM, CloudWatch logs, alarms, repository cleanup

Document map

Project overview	2
Week 1 - application and containerisation	3
Week 2 - AWS architecture and deployment	4-6
Week 3 - CI/CD automation	7
Week 4 - security, scaling, and monitoring	8-10
Cost estimation	11
Challenges and solutions	12-13
Conclusion and references	14

Configuration basis

The weekly progression is reconstructed from the repository history and current project files. Recommendations are labelled separately so planned controls are not presented as completed work.

SECTION 02

Week 1 - application and containerisation

Build a useful product locally, persist its data, and package it consistently

Week 1 established the product and the deployable artifact. The browser interface allows users to publish, browse, search, filter, edit, and delete skills, while the Express API performs validation and database operations. PostgreSQL stores the records so the platform behaves as a persistent application rather than a static demonstration.

Application request path



Area	Implementation	Outcome
Frontend	HTML5, CSS3, JavaScript	Skill form, search, filters, statistics, edit and delete interactions
Backend	Node.js 22 and Express 5	Serves static files and exposes POST, GET, PUT, and DELETE API routes
Persistence	PostgreSQL through pg	Creates the skills table when needed and stores application records
Configuration	dotenv and environment variables	Separates database host, port, name, username, and password from source code
Container	Docker on node:22-alpine	Copies dependencies and application files, exposes port 3000, runs server.js
Registry	Docker Hub	Publishes nomaseko97/skills-accelerator:latest for later AWS deployment

Week 1 build and verification

```
npm install
node app/server.js
docker build -t nomaseko97/skills-accelerator:latest .
docker run --env-file .env -p 3000:3000 nomaseko97/skills-accelerator:latest
```

- Create, read, update, and delete operations were exercised through the browser and REST API.
- Refreshing the application preserved stored records, confirming that PostgreSQL rather than browser memory held the data.
- The same container image became the release artifact used in Weeks 2 and 3.

Week 1 result

A complete local application and reproducible Docker artifact were ready before cloud infrastructure was introduced. This sequencing reduced AWS troubleshooting to infrastructure and configuration concerns instead of unfinished application logic.

SECTION 03

Week 2 foundation - complete AWS architecture

The Week 2 platform expanded by Week 3 automation and Week 4 operational services

Skills Accelerator — AWS project architecture

Terraform deployment | af-south-1

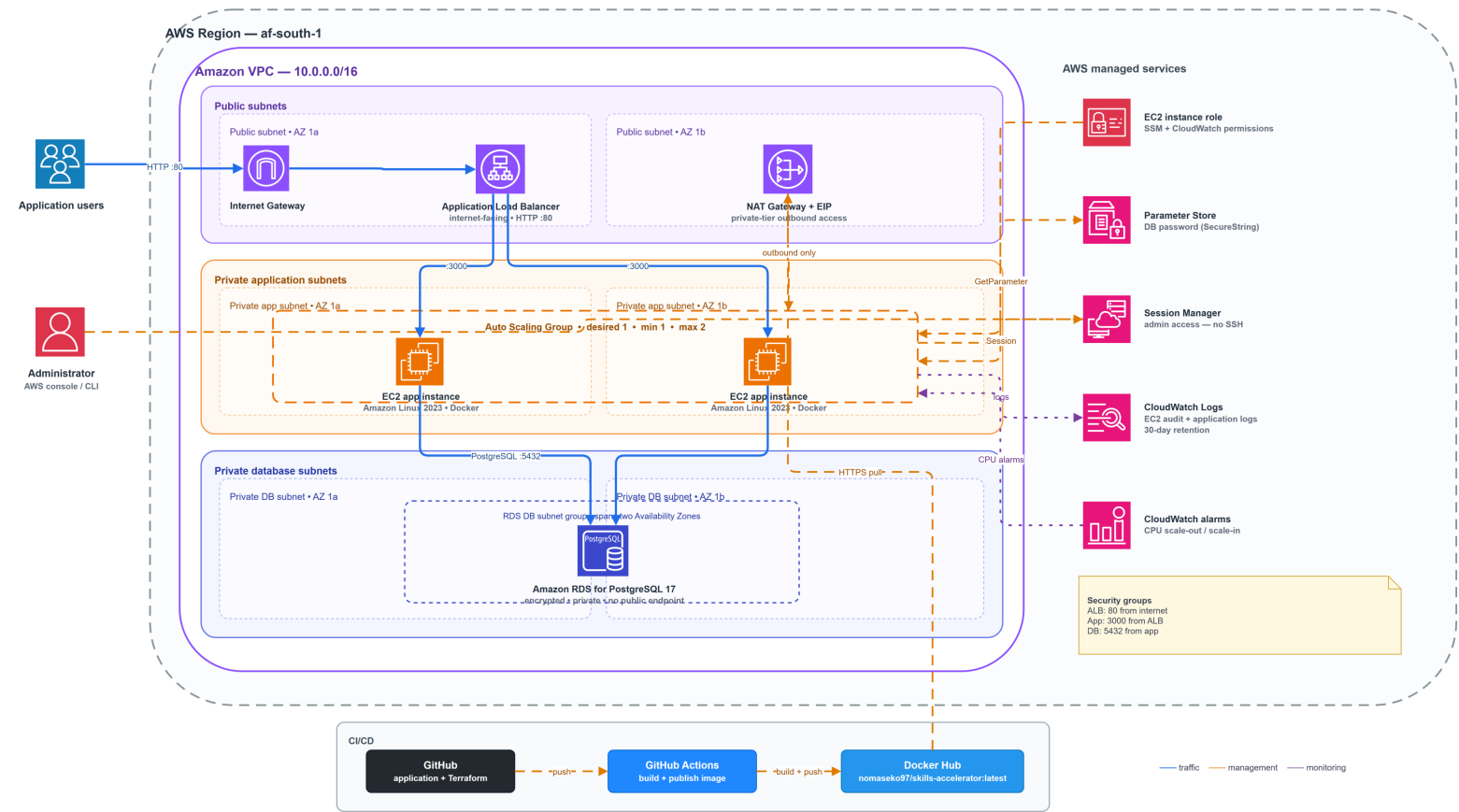


Figure 1. Complete logical architecture after Week 4. Week 2 created the VPC, ALB, private compute, and RDS foundation; Week 3 added automated delivery and CPU scaling; Week 4 added Parameter Store, scoped IAM, and centralized logging. The current Terraform uses two public and two private subnets.

SECTION 03

Week 2 - architecture and infrastructure

The deployed foundation and the supporting flows added in later weeks

Primary request path



Component	Configuration	Role
VPC	10.0.0.0/16	DNS support and hostnames enabled.
Public subnets	10.0.1.0/24 and 10.0.2.0/24	Host the internet-facing ALB; the NAT Gateway is in AZ 1a.
Private subnets	10.0.3.0/24 and 10.0.4.0/24	Host ASG instances and form the RDS DB subnet group.
Application Load Balancer	HTTP port 80	Forwards requests to healthy targets on port 3000.
Target group	HTTP port 3000	Health check GET / every 30 seconds; 2 healthy or unhealthy results.
EC2 application tier	Amazon Linux 2023, t3.micro	Docker runs the Node.js image in private subnets.
RDS data tier	PostgreSQL 17, db.t3.micro	20 GB, encrypted, private endpoint, single-AZ instance.
Outbound path	Private route -> NAT -> internet	Used for package updates, Docker Hub, and AWS public endpoints.

Supporting flows

- Administration: IAM-authorized operator -> Systems Manager Session Manager -> EC2. No SSH ingress rule is defined.
- Secrets: EC2 role -> ssm:GetParameter on /skills-accelerator/database/password -> decrypted value held in process memory and passed to the container.
- Observability: CloudWatch Agent sends auditd and cloud-init logs; the Docker awslogs driver sends application logs to the same log group.
- Scaling: CloudWatch CPU alarms invoke ASG policies; ALB health is also used for instance health decisions.

Availability interpretation
The ALB and subnet design span two Availability Zones, but desired capacity is one and RDS Multi-AZ is not enabled. The platform is multi-AZ-ready, not fully redundant in its current cost-optimized lab configuration.

SECTION 03

Week 2 - Terraform deployment

A repeatable infrastructure workflow with state and credential checks

Prerequisites

- Terraform CLI and AWS CLI installed on the workstation.
- An IAM principal with permission to manage the project resources and pass the EC2 instance role.
- AWS profile configured for af-south-1; verify the account before any plan or apply.
- A local terraform.tfvars created from terraform.tfvars.example. The real file must remain excluded from Git.

1. Configure and verify AWS access

```
aws configure --profile skills-accelerator-terraform
$env:AWS_PROFILE = 'skills-accelerator-terraform'
aws sts get-caller-identity --region af-south-1
```

2. Initialize, validate, plan, and apply

```
cd terraform
Copy-Item terraform.tfvars.example terraform.tfvars
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

3. Review outputs and validate AWS resources

- Record alb_dns_name, rds_endpoint, autoscaling_group_name, NAT public IP, VPC ID, and subnet IDs.
- Confirm the target group reports at least one healthy target before browser testing.
- Open http://<alb_dns_name> and test create, read, update, and delete operations.

State safety is mandatory

The configuration currently uses local state. A new computer therefore sees no managed resources unless the original state is restored or existing resources are imported. Never apply a plan showing a duplicate environment without first resolving state ownership. HashiCorp recommends remote state for shared work and locking [R2].

Existing-resource recovery

When a resource already exists outside the current state, confirm its identity and then use terraform import with the exact resource address and AWS identifier. Back up state before imports, run terraform plan after each import, and stop if Terraform proposes unintended replacement or duplication.

SECTION 04

Week 3 - CI/CD and deployment automation

Container build, registry publication, release delivery, and verification

Final EC2 bootstrap sequence

Week 3 automated image delivery and repeatable instance launches. The table shows the final bootstrap after Week 4 added CloudWatch collection and secure Parameter Store retrieval.

Step	Action	Implementation
1	Install	Update Amazon Linux; install Docker, CloudWatch Agent, and auditd.
2	Start services	Enable and start Docker and auditd.
3	Configure logs	Create the CloudWatch Agent JSON and start the agent.
4	Retrieve secret	Retry ssm:GetParameter up to 12 times with 10-second waits.
5	Identify instance	Use an IMDSv2 token to obtain the instance ID for log stream naming.
6	Run container	Pull the latest image and start it with DB variables and awslogs.

GitHub Actions workflow



The workflow logs in to Docker Hub using repository secrets, builds nomaseko97/skills-accelerator:latest, and pushes it. These build and publish stages match the platform design.

Current pipeline mismatch

The final workflow step deploys by SSH, but Terraform places EC2 in private subnets and defines no port 22 ingress. That step cannot reach a normal ASG instance directly. Replace it with Systems Manager Run Command, an ASG instance refresh, or a deployment service. Treat the current workflow as build-and-publish plus a legacy deploy step.

Post-deployment checks

- ALB returns the application and the target group remains healthy.
- CloudWatch log streams exist for audit, cloud-init, and application output.
- A Session Manager shell opens without SSH keys or inbound SSH access.
- The API returns persisted records after an instance replacement or container restart.

Safe teardown

```
terraform plan -destroy
terraform destroy
```

Confirm that the state is complete before destroying. The RDS resource uses `skip_final_snapshot = true`, so teardown deletes the database without a final snapshot. Export required data and capture evidence first.

SECTION 05

Week 4 - security and secrets management

Implemented controls, residual risks, and priority hardening actions

Control	Status	Evidence and interpretation
Network ingress	Implemented	ALB accepts TCP 80 from 0.0.0.0/0. App SG accepts 3000 only from ALB SG. RDS SG accepts 5432 only from app SG.
Private placement	Implemented	EC2 and RDS are not directly exposed to inbound internet traffic.
Administrative access	Implemented	Session Manager is enabled through the EC2 role; no SSH rule is present [R4].
Secret storage	Implemented	DB password is a SecureString encrypted with the default AWS managed KMS key [R3].
Secret authorization	Implemented	Inline policy allows only ssm:GetParameter on the single parameter ARN.
RDS at-rest encryption	Implemented	storage_encrypted = true; RDS encryption covers the DB storage [R6].
Database transport	Partial	The pg client enables SSL for RDS, but rejectUnauthorized = false disables certificate verification.
Public transport	Gap	The ALB listener is HTTP only. HTTPS, ACM certificate, and HTTP-to-HTTPS redirect are not configured.
Terraform state	Gap	The DB password remains present in Terraform state; local state must be treated as sensitive [R2].

Secret lifecycle



The change removes the password from EC2 user data and the Docker command embedded in Terraform. It does not remove the secret from Terraform state because Terraform manages both the RDS password and Parameter Store value.

Priority hardening actions

- Add HTTPS on the ALB with AWS Certificate Manager and redirect HTTP to HTTPS.
- Use the AWS RDS CA bundle and set rejectUnauthorized = true in the PostgreSQL client.
- Move state to an encrypted S3 backend with versioning and S3 lockfile support, or HCP Terraform.
- Consider a customer-managed KMS key or Secrets Manager when rotation and stronger separation are required.
- Pin container images by immutable tag or digest and add image vulnerability scanning.
- Set EC2 metadata_options.http_tokens = required to enforce IMDSv2, not only use it in the script.

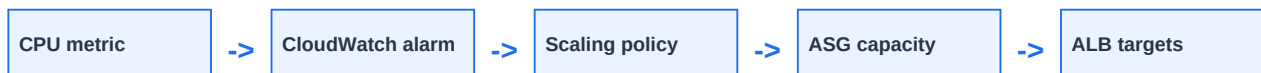
SECTION 04

Week 3 - scaling and resilience

CPU-based automation, replacement behavior, and a production path

MINIMUM 1 instance Cost floor	DESIRED 1 instance Normal capacity	MAXIMUM 2 instances Scale-out ceiling
SCALE OUT +1 CPU > 70%	SCALE IN -1 CPU < 30%	COOLDOWN 300 sec After each action

Decision flow



Signal	Condition	Evaluation	Result
High CPU	Average CPU > 70%	2 periods x 120 seconds	Add one instance
Low CPU	Average CPU < 30%	2 periods x 120 seconds	Remove one instance
Target health	HTTP GET / on port 3000	30-second interval; threshold 2	ALB routes only to healthy targets
ASG health	ELB health checks	120-second grace period	Replace failed instances

Current behavior and limitations

- At desired capacity one, the service can recover by replacement but cannot remain fully available during every instance or AZ failure.
- A maximum of two instances limits cost and is suitable for a demonstration workload, but provides little burst headroom.
- Simple scaling waits for the 300-second cooldown. AWS recommends target tracking or step scaling for better responsiveness [R7].
- The launch script performs package updates and image pulls, so instance warm-up can vary and should be measured.

Production progression

Use min = 2 and desired = 2 across both AZs, enable default instance warmup, move to target tracking (for example, 50-60% average CPU), use immutable launch template versions, and perform controlled instance refreshes. Enable RDS Multi-AZ only when availability requirements justify the additional database cost.

SECTION 05

Week 4 - monitoring and observability

What is collected today and how the operational signal should evolve

Log collection

Source	Log stream	Purpose	Publisher
/var/log/audit/audit.log	{instance_id}/audit	Linux security and audit events	CloudWatch Agent
/var/log/cloud-init-output.log	{instance_id}/cloud-init	Launch and bootstrap troubleshooting	CloudWatch Agent
Docker stdout/stderr	\$INSTANCE_ID/application	Node.js application and database messages	Docker awslogs

Retention

All streams use /skills-accelerator/ec2/audit with a 30-day retention policy. This provides a bounded evidence window and prevents indefinite storage growth.

Metrics and alarms

Signal	Condition	Action	Status
EC2 CPUUtilization	> 70%	Scale out +1	Implemented
EC2 CPUUtilization	< 30%	Scale in -1	Implemented
ALB target health	GET / fails	Stop routing / replace target	Implemented
UnHealthyHostCount	> 0	Notify operator	Recommended
HTTPCode_Target_5XX_Count	> baseline	Notify operator	Recommended
RDS FreeStorageSpace / CPU / connections	Capacity thresholds	Notify operator	Recommended

Operational observations

- The alarms invoke scaling policies but do not notify a person. Add an SNS topic and email or chat integration.
- The CloudWatch Agent configuration collects logs only; memory and disk metrics are not yet configured.
- EC2 detailed monitoring is not enabled. Two-minute alarm periods can receive sparse CPU data under five-minute basic monitoring; either enable detailed monitoring or use five-minute periods.
- The root path is used as the health endpoint. A dedicated /health route should check application readiness without performing expensive work.

Useful checks

```
aws logs tail /skills-accelerator/ec2/audit --follow --region af-south-1
aws elbv2 describe-target-health --target-group-arn <target-group-arn>
aws autoscaling describe-scaling-activities --auto-scaling-group-name skills-accelerator-asg
```

SECTION 06

Cost estimation

Illustrative on-demand monthly baseline for the complete platform in Africa (Cape Town)

Estimate date: 22 August 2026. The calculation uses 730 hours per month, one continuously running EC2 instance, a single-AZ RDS instance, 8 GB of EC2 gp3 root storage, 0.1 average ALB LCU, 5 GB of NAT processing, 1 GB of CloudWatch log ingestion and storage, two standard alarms, and three public IPv4 addresses (two ALB addresses plus the NAT EIP). Taxes, support plans, backups above the included allocation, cross-AZ traffic, and chargeable internet egress are excluded.

Service	Assumption	Monthly
EC2 t3.micro Linux	\$0.0136/hr x 730	\$9.93
EC2 gp3 root volume	\$0.1047/GB-mo x 8 GB	\$0.84
RDS db.t3.micro PostgreSQL	\$0.024/hr x 730	\$17.52
RDS General Purpose storage	\$0.151/GB-mo x 20 GB	\$3.02
Application Load Balancer	\$0.030/hr x 730	\$21.90
ALB capacity	\$0.0095/LCU-hr x 0.1 x 730	\$0.69
NAT Gateway	\$0.057/hr x 730	\$41.61
NAT data processing	\$0.057/GB x 5 GB	\$0.29
Public IPv4	\$0.005/hr x 3 x 730	\$10.95
CloudWatch log ingestion	\$0.678/GB x 1 GB	\$0.68
CloudWatch log storage	\$0.036/GB-mo x 1 GB	\$0.04
Two standard alarms	\$0.10/alarm-mo x 2	\$0.20

ESTIMATED BASELINE

USD 107.66 / month

NAT Gateway		\$41.90
Application Load Balancer		\$22.59
RDS PostgreSQL		\$20.54
EC2 + root EBS		\$10.77
Public IPv4		\$10.95

Cost interpretation

- The NAT Gateway is the largest fixed cost. Destroying idle lab environments produces the greatest saving.
- Running two EC2 instances for the full month adds about \$10.77, bringing the same baseline to about \$118.43 before extra traffic.
- Each additional NAT-processed GB adds \$0.057 before data transfer and possible cross-AZ charges.
- Actual ALB LCU, log volume, public IPv4 allocation, RDS backups, and internet egress determine the final bill.

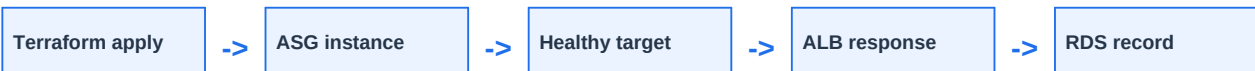
SECTION 07

Challenges and solutions - Weeks 1 and 2

Application, database, container, Terraform, and AWS infrastructure lessons

Challenge	Root cause	Resolution
Records did not persist without a database	A browser-only implementation could not preserve shared skill records between sessions.	Added PostgreSQL persistence, automatic skills-table creation, and CRUD operations through the pg client.
Local and container environments differed	The application depended on database settings and port configuration that vary by runtime.	Moved configuration to environment variables and packaged the runtime with a Dockerfile based on node:22-alpine.
Cloud creation was too manual	Creating networking, compute, load balancing, IAM, and RDS by hand was difficult to repeat safely.	Defined the environment in Terraform with variables, outputs, and reusable resource relationships.
Private EC2 needed administration	The secure design removed public exposure and did not include inbound SSH access.	Attached an IAM instance profile and used Systems Manager Session Manager for controlled administration.
RDS connection failed without SSL	The hosted PostgreSQL endpoint required an encrypted client connection.	Enabled SSL for RDS connections in db.js; the next step is validation with the AWS RDS CA bundle.
IAM PassRole was denied	The deployment identity could create resources but could not assign the EC2 role to instances.	Added the necessary iam:PassRole permission for the specific project role and verified the launch template.

Week 2 verification chain



A running instance was not enough evidence

The deployment was considered successful only when the target group was healthy, the application loaded through the ALB, the API returned expected data, and a record remained available in RDS. This end-to-end test connected the Week 1 application to the Week 2 infrastructure.

Lesson

Each layer should be validated before the next is blamed: application behavior, container runtime, security groups, load-balancer health, instance bootstrap, and database connectivity. The architecture became dependable because verification followed the same path as a real user request.

SECTION 07

Challenges and solutions - Weeks 3 and 4

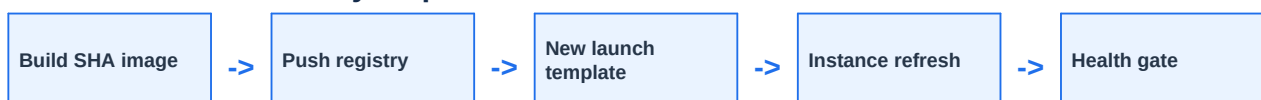
Automation, access control, monitoring, state recovery, and operational lessons

Challenge	Root cause	Implemented or recommended solution
CI/CD deploy path conflicts with private EC2	The workflow uses SSH, while the ASG has no public instances and no port 22 rule.	Keep build and push; replace remote deployment with SSM Run Command or an ASG instance refresh.
Mutable latest tag	Every instance pulls the same mutable image tag, reducing deployment traceability and rollback certainty.	Publish a commit-SHA tag, reference an immutable digest, and retain a tested previous version.
Database password appeared in user data	The launch template originally interpolated <code>var.db_password</code> directly into the Docker command.	Created a <code>SecureString</code> parameter and changed EC2 bootstrap to retrieve it with a resource-scoped role.
Sensitive tfvars was tracked	A real <code>terraform.tfvars</code> file had entered version control history.	Removed it from tracking, added ignore rules, and supplied a safe <code>terraform.tfvars.example</code> . Rotate any exposed credential.
IAM policy and import errors	The provisioning user lacked SSM, Logs, and IAM read actions, while inline policies reached the 2,048-character quota.	Added narrowly required lifecycle actions through a customer-managed policy; retained least privilege for the EC2 runtime role.
New computer had code but not state	Git contained Terraform configuration but correctly excluded the sensitive local <code>terraform.tfstate</code> file.	Backed up partial state, imported verified resources, and identified remote encrypted state with locking as the permanent fix.

Engineering lessons

- Separate provisioning permissions from runtime permissions. The IAM user needs lifecycle actions; EC2 should receive only the permissions the workload requires.
- Treat state, plans, tfvars, access keys, and application environment files as sensitive artifacts with different lifecycle controls.
- Prefer deployment mechanisms that understand Auto Scaling. Updating one instance by SSH does not update future replacement instances.
- Verify success at every layer: Terraform plan, AWS resource health, container status, application response, database persistence, logs, and replacement behavior.
- Document the difference between a cost-optimized demonstration architecture and a production availability posture.

Recommended delivery sequence



Decision point before destroy

Destroy only when the active state is known to represent the complete environment and all required evidence or data has been preserved. A partial state can leave billable resources behind; an incorrect state can target the wrong resources.

SECTION 08

Conclusion and next steps

Four-week achievement summary, prioritized roadmap, and source references

Across four weeks, the project developed as one continuous system. Week 1 delivered the functional CRUD application, PostgreSQL persistence, and Docker image. Week 2 provisioned the AWS network, load balancer, private Auto Scaling compute, managed RDS database, IAM role, and Session Manager access with Terraform. Week 3 automated image build and publication, introduced the deployment workflow, and added CPU-driven scaling. Week 4 removed the database password from EC2 user data, applied resource-scoped runtime IAM, centralized logs, strengthened repository hygiene, and documented the operational recovery process.

Prioritized roadmap

Priority	Work item	Outcome
P0	Remote Terraform state	Encrypted S3 backend with versioning and use_lockfile, or HCP Terraform.
P0	Secure public traffic	ACM certificate, HTTPS listener, and HTTP redirect.
P0	Fix deployment mechanism	Replace SSH with SSM Run Command or ASG instance refresh.
P1	Database trust	Use the AWS RDS CA bundle and verify the server certificate.
P1	Actionable monitoring	SNS notifications plus ALB, RDS, memory, disk, and deployment alarms.
P1	Higher availability	Two baseline EC2 instances; consider RDS Multi-AZ if justified.
P2	Supply-chain control	Immutable image tags or digests, scanning, and tested rollback.

References

Ref	Source
R1	HashiCorp Terraform CLI workflow
R2	HashiCorp remote state and sensitive data
R3	AWS Parameter Store SecureString encryption
R4	AWS Systems Manager Session Manager
R5	Amazon CloudWatch Agent
R6	Amazon RDS encryption
R7	EC2 Auto Scaling cooldowns
R8	ALB target health checks
R9	AWS Price List - Amazon EC2, Cape Town
R10	AWS Price List - Amazon RDS, Cape Town
R11	AWS Price List - ELB, VPC, and CloudWatch

Final assessment

The platform is a strong demonstration of secure, scalable AWS application delivery. Its next maturity step is not adding more services; it is tightening state management, transport security, deployment consistency, and alerting so the implemented controls operate as one dependable system.